MODULE II • LECTURE 05

Convolutional Networks for Text Classification

Convolution • Filters • Feature Maps • Pooling • Sentence Classification

Dr Tamal Ghosh

Recap: Feedforward Networks

What Can They Do?

A feedforward network learns transformations of text representations:

$$x \rightarrow W_1 x + b_1 \rightarrow f \rightarrow W_2 a + b_2 \rightarrow \hat{y}$$

It acts as a powerful non-linear classifier.

? The Limitation

When feeding a full sentence into a dense network, it typically views the input globally.

> **Question:**
> Does a fully connected network naturally understand *local* word sequences and phrases?

Learning Objectives

By the end of this lecture, you should be able to:

- Explain the basic idea of CNNs.
- Explain convolution in the context of text.
- Understand filters and feature maps.
- Calculate a simple 1D convolution.
- Explain padding and stride.
- Understand max pooling.
- Explain how CNNs capture local n-gram-like patterns.
- Design a simple CNN for text classification.
- Understand the advantages and limitations of CNNs for NLP.

Why CNNs for Text?

Language is Local

Language contains important local patterns that define meaning regardless of the surrounding context.

```
"very good movie"

"not at all useful"

"extremely disappointing"
```

The Semantic Signal

These short sequences can carry strong semantic signals on their own.

CNNs are exceptionally capable architectures for detecting:

Local Patterns

The Core Idea

A CNN applies a small **learnable filter** across the input.

For text, it slides across the sequence of words:

Word 1 Word 2 Word 3 Word 4

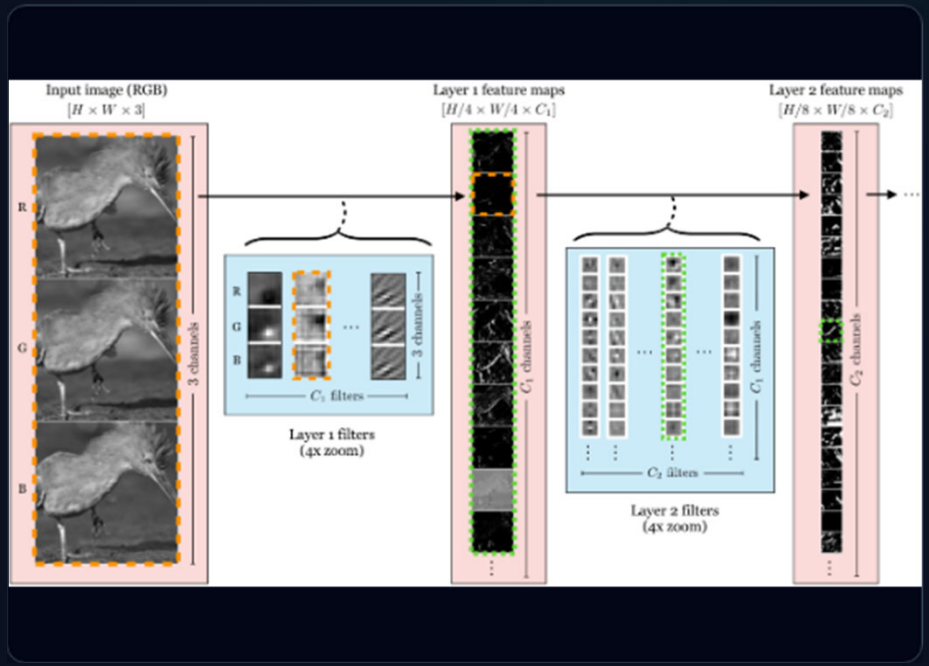
↓

Sliding Filter

↓

Feature Map

The same filter can detect a pattern wherever it occurs in the sentence.



CNNs in Computer Vision vs NLP

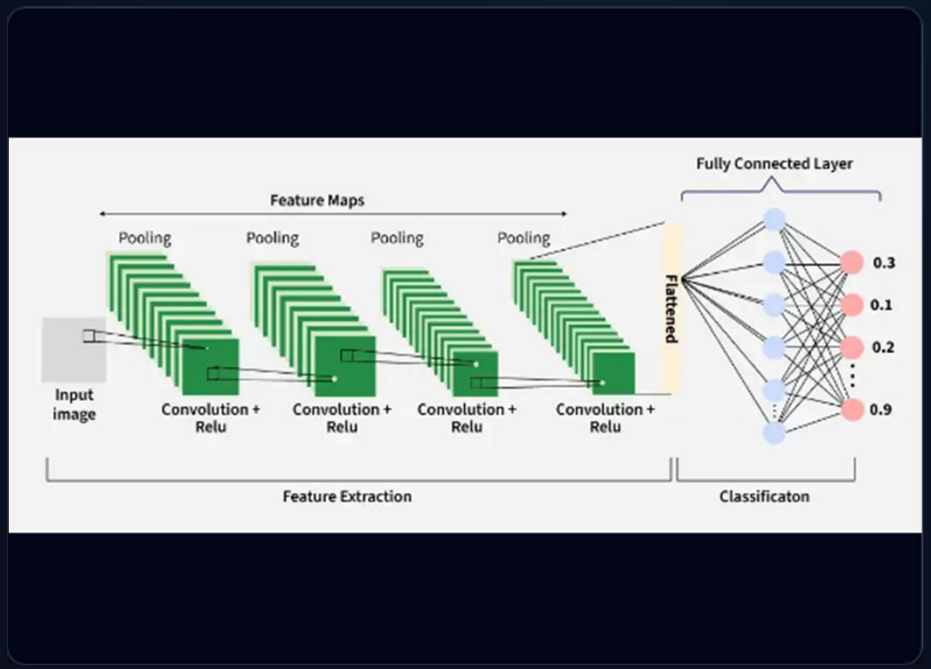
Image Classification:

- 2D Image Input
- 2D Convolution (Height × Width)
- Generates 2D Feature Maps

Text Classification:

- Sequence of word vectors
- 1D Convolution (Across sequence length only)
- Generates 1D Feature Maps

> **The Principle is Identical:**
> Learn filters that detect useful local patterns.



Sentences must be converted into numerical matrices.

Suppose our input is:

```
"This movie is really good"
```

We represent each word as a vector:

This → e_1
movie → e_2
is → e_3
really → e_4
good → e_5

The Input Matrix

Stacking these vectors creates a matrix:

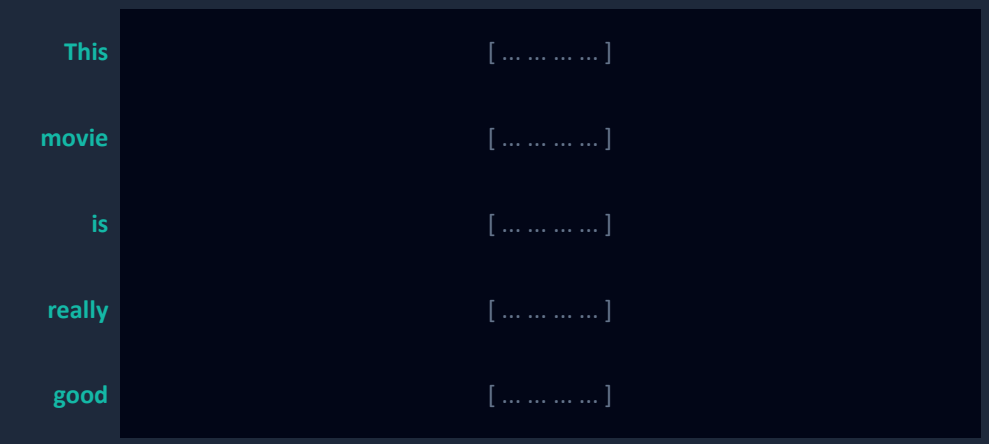
$$X \in R^{L \times d}$$

L = number of tokens (sequence length)

d = embedding dimension

The Sentence Matrix

For a five-word sentence, the matrix looks like this:



Rows represent tokens. Columns represent dimensions.

This matrix becomes the CNN input.

What Is a Filter?

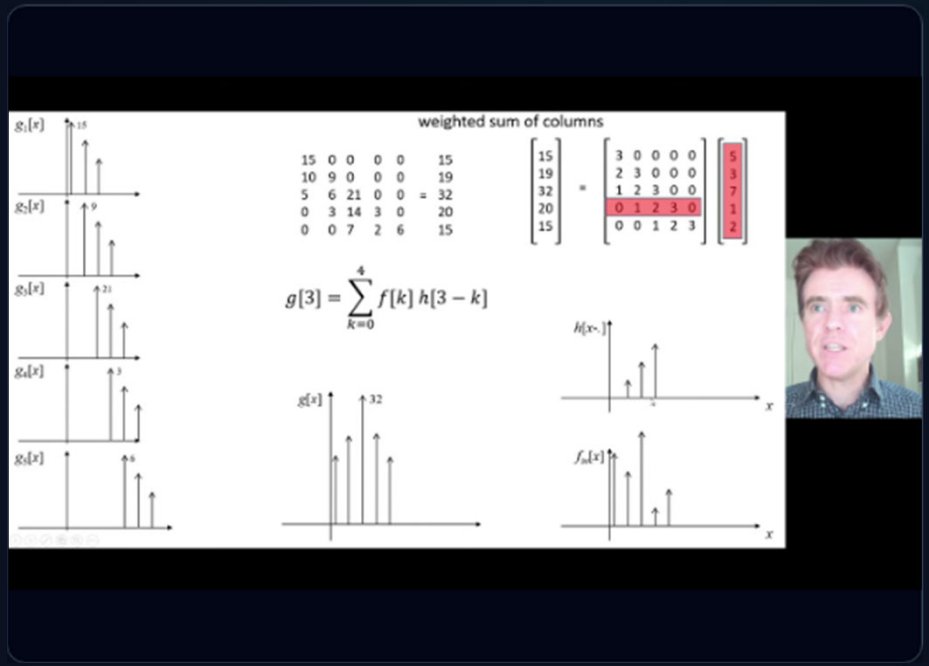
A filter is a small set of **learnable weights**.

A filter with a height of 3 (window size 3) examines 3 words at a time. It slides down the sequence:

```
[Word 1, Word 2, Word 3] → output
1

[Word 2, Word 3, Word 4] → output
2

[Word 3, Word 4, Word 5] → output
3
```



The exact same filter is reused at every position.

Why Reuse the Same Filter?

Detecting Patterns

Suppose a filter learns to respond strongly to a pattern resembling:

```
"not + very + good"
```

It should be able to detect this pattern whether it appears at the beginning, middle, or end of the document.

Parameter Sharing

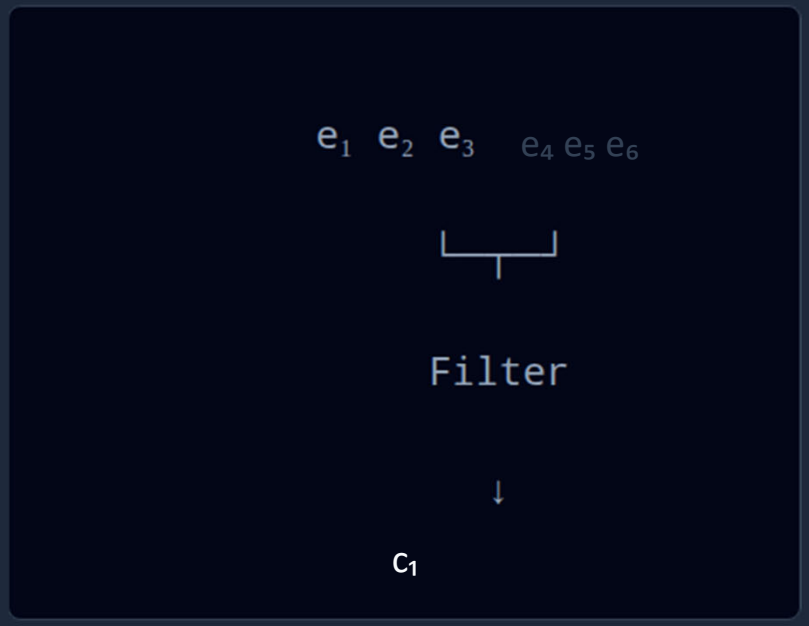
Reusing the filter ensures **positional invariance**.

This is called **Parameter Sharing**. The same filter parameters are applied across all positions, making the model highly efficient.

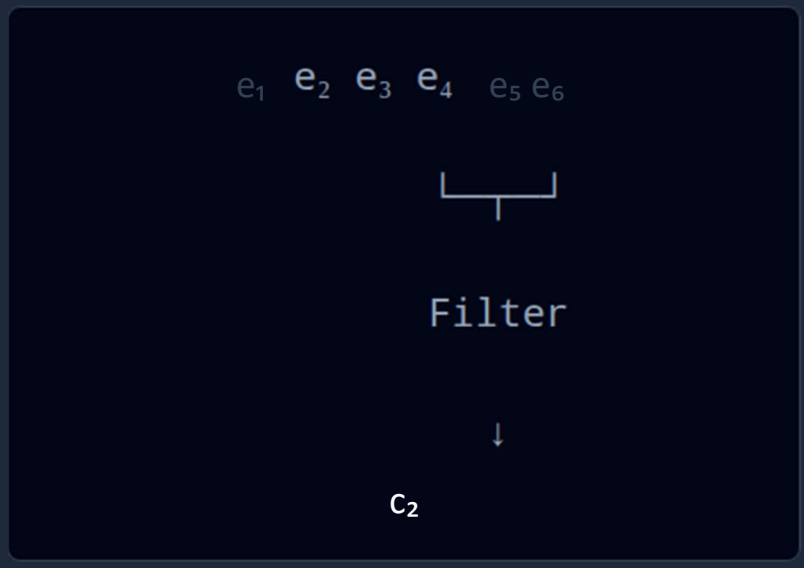
1D convolution for text

For text, we commonly use a 1D convolution over the sequence dimension.

Position 1:



Position 2:



The filter generates a sequence of activation values

Mathematical View

The Dot Product

For a local region X_i, a filter computes a linear combination:

$$z_i = W \cdot X_i + b$$

followed by a non-linear activation (e.g., ReLU):

$$c_i = f(z_i)$$

The Feature Map

Applying this to all valid sliding windows produces a sequence:

$$c = [c_1, c_2, \dots, c_n]$$

This sequence is the Feature Map.

Simple Numerical Convolution (1/2)

Let's walk through a simplified 1D sequence manually.

Inputs

Consider a simplified sequence:

$$X = [2, 1, 3, 4, 2]$$

and a filter of size 3:

$$W = [1, -1, 1]$$

First Window

For the first window , we calculate the dot product:

$$2(1) + 1(-1) + 3(1) = 4$$

So: $= 4$

Sliding the Filter (2/2)

Second Window

Slide to the right: $[1, 3, 4]$

$$1(1) + 3(-1) + 4(1) = 2$$

So: $c_2 = 2$

Third Window

Slide again: $[3, 4, 2]$

$$3(1) + 4(-1) + 2(1) = 1$$

So: $c_3 = 1$

$$c = [4, 2, 1]$$

This is our pre-activation feature map.

Activation After Convolution

Applying ReLU

Usually convolution is followed by an activation function like ReLU.

$$\text{ReLU}(z) = \max(0, z)$$

Example

If a filter outputs negative values indicating the pattern wasn't found:

$$c = [-2, 4, -1, 3]$$

Then after ReLU:

$$\text{ReLU}(c) = [0, 4, 0, 3]$$

The network retains only positive pattern responses.

What Does a Filter Learn?

A CNN filter may learn to respond to linguistic patterns associated with:

- Negation ("not useful")
- Strong sentiment ("extremely disappointing")
- Intensifiers ("very good")
- Short phrases ("poor quality")
- Syntactic fragments
- Domain-specific expressions

Emergent Behavior

We do not explicitly tell the filters what to look for.

The exact learned pattern is determined entirely during training via backpropagation.

CNN as Learned N-gram Detection

Filter widths correspond directly to n-gram sizes in traditional NLP.

Width 2

A filter of width 2 inspects approximately **2-token patterns** (Bigrams).

e.g., "not good"

Width 3

A filter of width 3 inspects approximately **3-token patterns** (Trigrams).

e.g., "very well made"

Width 5

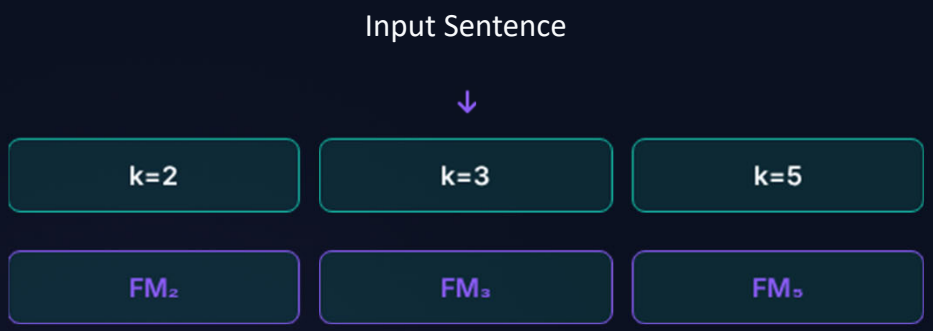
A filter of width 5 inspects approximately **5-token patterns**.

e.g., "one of the best movies"

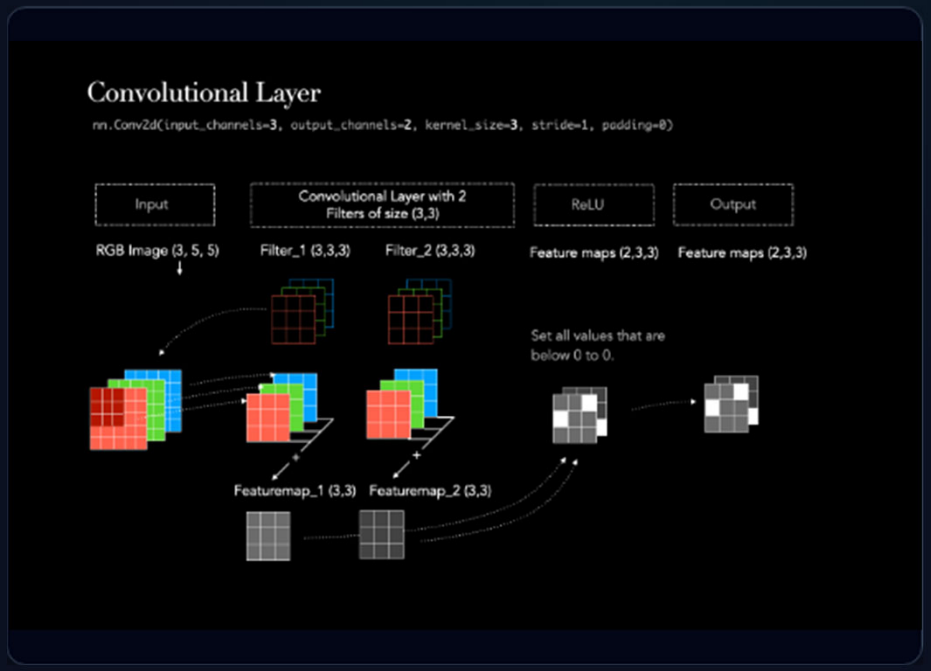
This is why CNNs are often described as learning n-gram-like local features.

Multiple Filter Sizes

To capture patterns of varying lengths, a text CNN typically uses multiple kernel (filter) sizes in parallel.



Different filter widths capture patterns at different local scales.



Each filter generates one specific feature map across the text.

Filter 1 (e.g., Sentiment Intensifiers):

```
[0.1, 0.8, 0.2, 0.0, 0.7]
```

Filter 2 (e.g., Negation):

```
[0.0, 0.1, 0.9, 0.4, 0.2]
```

Filter 3 (e.g., Punctuation patterns):

```
[0.5, 0.3, 0.1, 0.8, 0.1]
```

Each map represents the **response intensity** of one learned detector across the sequence.

Pooling

Summarizing the most important patterns.

Why Do We Need Pooling?

The Problem

A feature map contains multiple activations across the sentence.

$$c = [\, 0.1 , 0.8 , 0.2 , 0.4 , 0.9 \,]$$

For document-level classification, we don't necessarily care exactly *where* the pattern was; we care *if* it was there.

The Solution

We want to summarize the strongest evidence found by the filter.

This motivates the use of:

Max Pooling

Max Pooling

Given the feature map:

$$c = [\, 0.1 , 0.8 , 0.2 , 0.4 , 0.9 \,]$$

Global max pooling extracts the maximum value:

$$\max(c) = 0.9$$

Interpretation:
The strongest response of this specific filter *anywhere* in the sentence is 0.9.

Why Max Pooling Works for Text

Position Independence

Suppose a filter successfully learns to detect the word **"excellent"**.

The exact position of "excellent" in the sentence usually does not matter for determining overall sentiment.

The Binary Signal

We simply want the network to answer one question for that filter:

"Did this important pattern appear anywhere in the text?"

Max pooling provides exactly this simple, summarized answer.

Global Max Pooling Operation

For each feature map : c_j

$$c_j = [\, c_{j1} , c_{j2} , \cdots , c_{jn} \,]$$

compute the max:

$$p_j = \max_i c_{ji}$$

If we have 100 filters, we get 100 max values:

$$p = [\, p_1 , p_2 , \cdots , p_{100} \,]$$

The variable-length sequence has now been converted into a fixed-length vector.

Why Fixed-Length Output?

Variable Inputs

Sentences have different lengths.

```
"Short sentence"
```

versus

```
"This is a considerably longer sentence containing many more words."
```

The Classifier's Need

A standard dense classifier layer needs a fixed-size input vector.

Variable-length maps

↓

Global Max Pooling

↓

Fixed-length vector

Basic CNN Text Classifier

The Classic Architecture:

Text

↓

Word Embeddings

↓

Convolution (Multiple k)

↓

ReLU

↓

Max Pooling

↓

Concatenation

↓

Fully Connected Layer

↓

Output

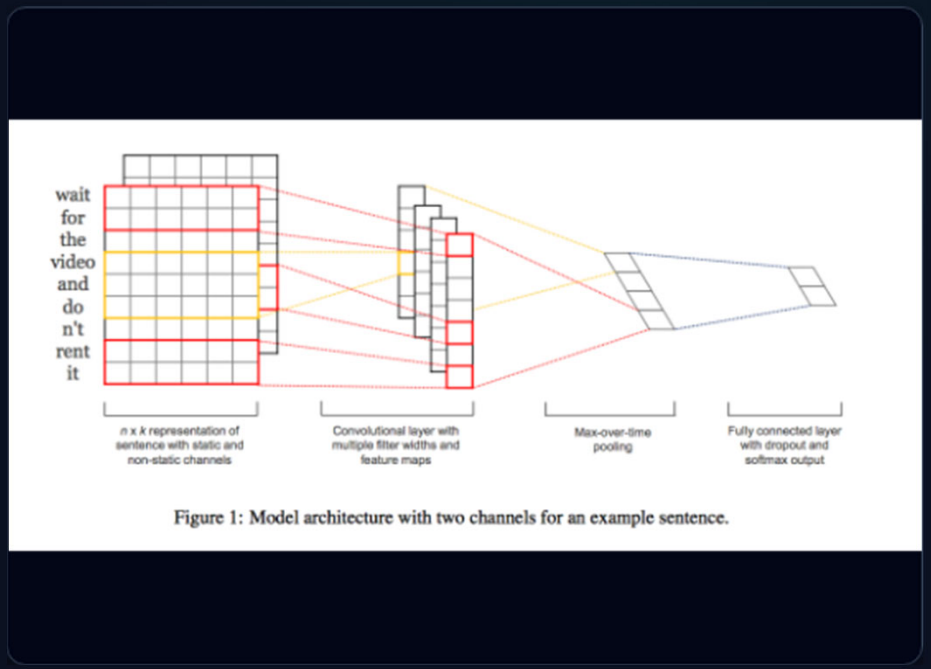
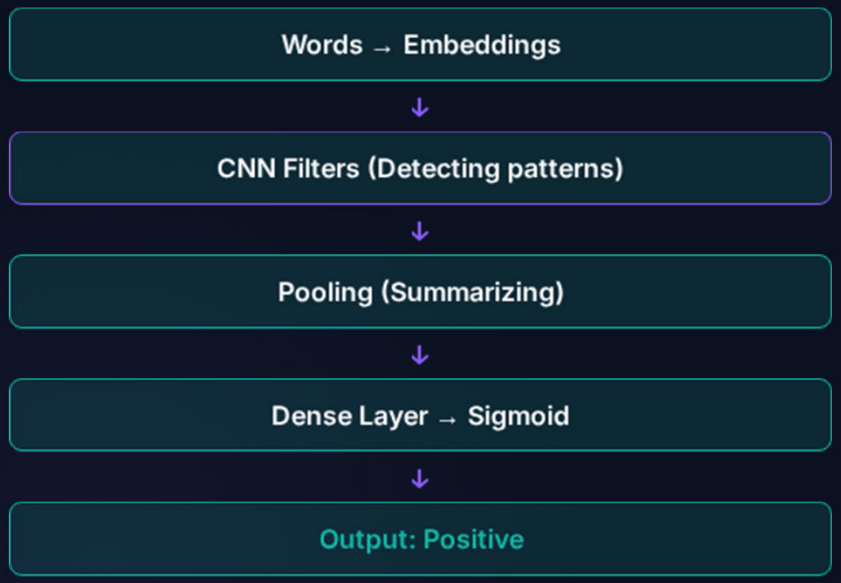


Figure 1: Model architecture with two channels for an example sentence.

Sentiment Analysis Example

Input: "The movie was surprisingly good."



What happens inside?

Different filters may respond highly to specific parts of the sentence:

- **Filter A:** "surprisingly good"
- **Filter B:** "movie was" (context)
- **Filter C:** General positive modifiers

Advantages of CNNs for NLP

- **Efficient parallel computation:** Convolutions can be heavily optimized on GPUs.
- **Local feature extraction:** Excellent at finding n-gram patterns.
- **Parameter sharing:** Highly efficient use of weights compared to dense networks.
- **Positional invariance:** Ability to detect patterns regardless of exact position.
- **Multiple scales:** Multiple kernel sizes easily capture different phrase lengths.
- Strong baseline performance on many classification tasks.

CNNs are especially useful when local patterns (phrases/n-grams) are the most important indicators of the target label.

Limitations of CNNs for NLP

Local Receptive Field

A basic CNN initially focuses only on local regions (windows of 3-5 words).

Long-Range Dependencies

Relationships between distant words may be much harder to capture.

Example Failure Case

```
"The movie, despite several problems in the middle, was excellent."
```

The relevant semantic relationship ("movie... excellent") spans a large portion of the sentence, separated by a contrasting clause.

CNNs are not automatically ideal for every NLP problem.

CNN vs Feedforward Network

Property	Feedforward NN	CNN
Main operation	Dense transformation	Convolution
Local patterns	Not explicitly emphasized	**Strong**
Parameter sharing	Usually no	**Yes**
Sequence position	Flattened / encoded	Preserved locally
Variable-length text	Requires rigid preprocessing	**Pooling can handle it**
Typical NLP use	General classification	Text classification, local pattern detection

Key distinction: CNNs build locality and parameter sharing directly into the architecture.

Summary & Next Lecture

Key Takeaways

- **CNNs:** Detect local patterns efficiently.
- **Filters:** Learn reusable pattern detectors.
- **Feature maps:** Record responses across sequence.
- **Pooling:** Summarizes important activations.

Embed → Conv → ReLU → Pool → Classify

Next Lecture

Applications of Deep Learning in NLP

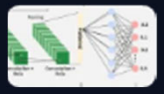
Sentiment Analysis & Word Sense Disambiguation

We will move from architecture to real NLP applications and examine how neural models are actually used to solve linguistic problems.

Image Sources

https://visionbook.mit.edu/figures/convolutional_neural_nets/alexnet_feature_maps.png

Source: visionbook.mit.edu



https://media.geeksforgeeks.org/wp-content/uploads/20260217113409515144/working_of_cnn__.webp

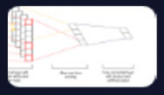
Source: www.geeksforgeeks.org

https://i.ytimg.com/vi/W2_nD85jL5s/maxresdefault.jpg

Source: www.youtube.com

https://miro.medium.com/v2/resize:fit:2000/1*QLzgMSCIaZFZUrrGwiId_w.png

Source: medium.com



https://cezannec.github.io/assets/cnn_text/complete_text_classification_CNN.png

Source: cezannec.github.io